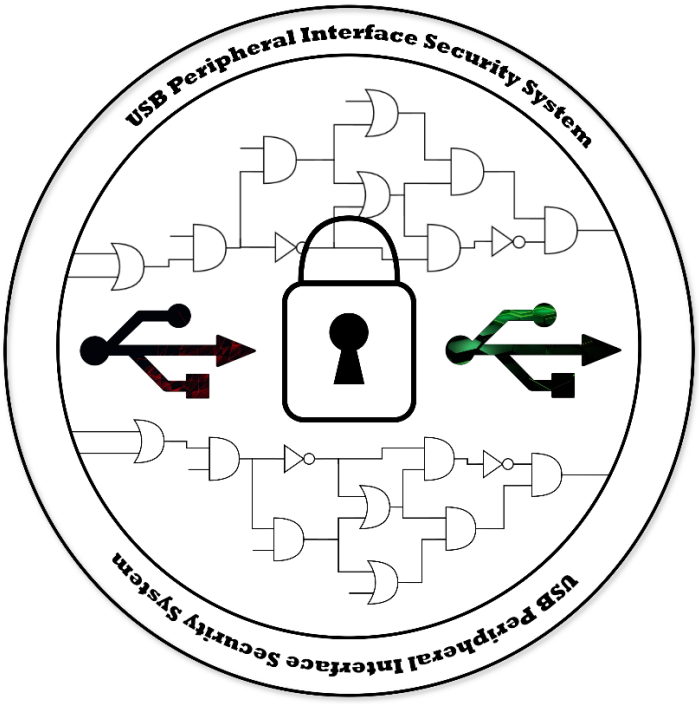


USB Peripheral Interface Security System

Group 19 – Fall 26'/Spring 27' ECE Senior Design

University of Central Florida

Authors

Ryan Eng *(CpE)* | Kyle Kosuda *(EE)* | Victor Valentin *(CpE)* | Lukas Zavaleta *(CpE)*

Review Committee

Dr. Mike Borowczak	Associate Professor
Dr. Chris Ianello	Professor of Practice
Dr. Fan Yao	Associate Professor

Advisor

Dr. Suboh Suboh	Associate Lecturer

Table of Contents

1 Executive Summary (Placeholder) 1
2 Project Description 1
2.1 Motivation and Background 1
2.2 Past Work 2
2.3 Goals and Objectives 3
2.3.1 Goals 3
2.3.2 Objectives 4
2.4 Features and Functionalities 4
2.4.1 Threat Model 5
2.5 Engineering Specifications and Requirements 6
2.5.1 System Specifications and Requirements 6
2.5.2 Component Specifications and Requirements 7
2.6 Hardware Block Diagram 7
2.7 FPGA Fabric Block Diagram 9
2.8 Embedded Software Diagram 10
2.9 Prototype 11
2.10 House of Quality 13
2.11 Budget and Finances 14
2.12 Milestones and Timeline 15
2.13 Risk Analysis and Mitigation 16
3 Appendices ..... A
3.1 Appendix A – References ..... A
3.2 Appendix B – Copyright Permission ..... A
3.3 Appendix C – Etc. ..... A

Table of Figures

Figure 1: Main Hardware Block Diagram 8
Figure 2: FPGA Block Diagram 9
Figure 3: Software Flowchart 10
Figure 4: External Prototype Illustration 11
Figure 5: Internal Prototype Illustration 12
Figure 6: GUI Prototype 12
Figure 7: House of Quality 13

Table of Tables

Table 1: Overall System Specifications 6
Table 2: Component Specifications 7
Table 3: Bill of Materials 14
Table 4: Milestones and Timeline 15
Table 5: Risk Analysis 16

Table of Acronyms

Acronym	Definition
AC	Alternating Current
CpE	Computer Engineering
DoS	Denial of Service
ECE	Electrical and Computer Engineering
EE	Electrical Engineering
ESD	ElectroStatic Discharge
FPGA	Field Programmable Gate Array
GDT	Gas Discharge Tube
GUI	Graphical User Interface
HID	Human Interface Device
HTTPS	HyperText Transfer Protocol Secure
IC	Integrated Circuit
LDO	Low DropOut
LUT	LookUp Table
MCU	MicroController Unit
OS	Operating System
PC	Personal Computer
PCB	Printed Circuit Board
PHY	Physical Layer
PIN	Personal Identification Number
PPTC	Polymeric Positive Temperature Coefficient
R/W	Read/Write
SD	Senior Design
SMT	Surface Mount Technology
SPI	Serial Peripheral Interface
TBU	Transient Blocking Unit
TVS	Transient Voltage Suppressor
UART	Universal Asynchronous Reception and Transmission
USB	Universal Serial Bus
USG	USB Security Gateway
VID	Vendor Identification

1 Executive Summary (Placeholder)

2 Project Description

2.1 Motivation and Background

In the current day, malicious cyber-attacks have become more prevalent than ever, contributing to the loss or theft of data, destruction of property, and compromise of important systems. One such system that malicious parties can exploit to cause such damage is via the Universal Serial Bus (USB) port, commonly used on many everyday personal computing devices.

USB is one of the most widely used interfaces for connecting peripherals to computers, electronics, and many embedded systems. It is a simple and convenient connection, one that allows devices to be powered, programmed, and interfaced simply by plugging them in. However, its popularity and widespread use make it a prime target for attacks. USB peripherals cannot be assumed to be inherently trustworthy solely because they follow USB protocols. Hosts often communicate with devices for initialization before their behavior is fully evaluated, leaving them vulnerable to attack.

Many USB attacks can be adequately described as a "trojan horse" because they require an individual to connect a malicious device to their trusted host device to properly function. Examples of such attacks include “USB Killer” and “BadUSB” attacks. “USB Killer” attacks use the 5V connection to rapidly charge and discharge over 240V of power into the hardware [1]. This can cause permanent damage to a system and potentially render a system unusable. “BadUSB” attacks involve HID spoofing, a method in which a malicious USB device masquerades as a legitimate peripheral such as a keyboard. The device can then open sensitive applications such as the command prompt at quick speeds and cause damage to the system. Users, however, cannot be trusted to properly verify a device before connecting it. One publication finds that an attack relying on users to pick up dropped USB devices has an estimated success rate of 45-98% [2]. Adequate hardware and software protection can provide the means to eliminate users from inadvertently compromising a system.

The USB protocol specification does not list security or device safety as a design goal or feature in the base USB 2.0/3.2 protocol [3]. Its core objective is designed almost exclusively for ease of peripheral connection, low overhead cost, power efficiency, and high performance. Security checks are often deferred to higher layers in communication; implemented in operating system (OS) software, upper layer protocols such as HTTPS, or internal host device hardware. While host-based security is sufficient for responding to certain threats, it places the trusted system in direct contact with potentially hostile hardware. This presents the need for hardware-level security boundaries that maintain a gateway between a potentially dangerous USB peripheral and the trusted host.

A typical USB hub provides an easy method of connectivity and basic electrical protection for a trusted host. Their intended purpose is to moderate and distribute USB communications to multiple devices; they do not analyze or enforce security measures.

This leads to cases where a user may inadvertently connect to a malicious USB device which is given access to the trusted host and other USB peripherals [2]. An intentionally malicious device can leverage device impersonation, false enumeration, protocol abuse, and electrical faults to cause temporary or permanent damage to device hardware, inject malware into a host device, or compromise sensitive data within a system [4].

As security becomes a more prevalent concern in device hardware, more strategic means are required to defend devices from malicious attacks. This project proposes to combine the utility of a common USB hub with the defense of a hardware-based security gateway to create a four-port USB expansion hub capable of preventing electrical damage or malicious attacks via the USB port of a device. A Field Programmable Gate Array (FPGA) architecture will be leveraged to monitor USB communications and provide hardware-level mechanisms for identifying and responding to potentially malicious behavior. Additionally, each downstream port will feature independent power control and analog electrical protection circuitry to isolate and quarantine compromised peripherals, without disabling the remaining ports.

This project proposes a system which demonstrates a hardware-centric approach to securing a ubiquitous and inherently vulnerable communication interface. Dedicated hardware boundaries can provide an additional layer of security that software environments are unable to implement. This project aims to demonstrate that a USB hub can serve not only as a connectivity device, but also as an active security boundary capable of monitoring, controlling, and isolating potentially untrusted peripherals.

2.2 Past Work

Several existing security systems have explored methods of protecting computers from malicious or unauthorized USB devices. These existing systems are useful for our project because they demonstrate different approaches to hardware isolation, USB device identification, and real-time traffic monitoring.

In our research, we found that USB Sentry is one of the closest existing products to our project. USB Sentry combines a four-port USB hub with a hardware firewall that monitors connected devices and can prevent unauthorized USB behavior from reaching the host. This is comparable to our project because both systems provide multiple downstream USB ports, with security hardware between the peripherals and the host. The main difference is that USB Sentry uses microcontrollers for its firewall functionality, while our project plans to use an FPGA for real-time USB monitoring [5].

Another relevant technology is USBGuard, a software framework for controlling which USB devices are allowed to communicate with the computer. USBGuard makes authorization decisions using information such as device identifiers and the classes of interfaces exposed by a USB device [6]. This is relevant to our requirement to monitor USB enumeration and identify the device and interface classes of each connected downstream device. However, our project places the security mechanism in external hardware before USB traffic reaches the host, while USBGuard operates as software on a host.

The USB Security Gateway (USG) provides another example of hardware-based USB protection. USG is placed between an untrusted USB peripheral and the host and uses separate processors to restrict the USB communication that is allowed to cross the security boundary [7]. Our project architecture follows the same general idea of physically isolating an untrusted USB device from the host. Unlike USG, our project will support four independently controlled downstream USB ports and will use an FPGA as part of the security path.

Reviewing these existing systems gives our team a better understanding of the different approaches that can be used to secure USB devices. USB Sentry demonstrates a multi-port hardware firewall; USBGuard shows how device information can be used for authorization, and USG demonstrates hardware isolation between a peripheral and host. Our project takes ideas from these approaches while using a four-port design with an FPGA for USB monitoring and a microcontroller for system control.

2.3 Goals and Objectives

In this section are the goals and objectives that were established for the Four-Port USB Hub Security Gateway. Our goals describe the broader idea and vision for the project and its outcome, while our objectives describe specific deliverables and demonstratable capabilities that define the systems' implementation. Each category is divided into basic, advanced, and stretched goals and objectives. Stretch goals and objectives are desired and beneficial to the project but may be beyond the scope of the project's boundaries and implementation.

2.3.1 Goals

Basic Goals:

- Create a compact and enclosed device capable of moderating four downstream USB ports into one upstream port
- Provide full USB 2.0 protocol functionality to all downstream ports including appropriate power availability and data communication
- Protect hardware from typical electrical surges and faults
- Provide monitoring of USB protocol messages for scanning potential threats
- Provide swift, automatic responsiveness to identified potential threats
- Provide a method of viewing device status and security logs
- Isolate USB ports identified as potential threats

Advanced Goals:

- Provide robust transient voltage protection on each USB port
- Provide protection against potential threats that may inject keystrokes or electrically damage internal hardware
- Create a fully interactable PC GUI showing connected devices, security status, and event logging
- Provide protection against potential threats which may create a denial-of-service interruption

Stretch Goals:

- Provide full USB 3.2 protocol functionality to all downstream ports
- Allow for user redefinition of security parameters

2.3.2 Objectives

Basic Objectives:

- Design a compact PCB with a USB Hub IC capable of moderating USB four to one communication
- Design circuitry with properly routed data lines and provide power from host to downstream USB ports
- Provide a method of providing auxiliary power through a barrel jack connector to ensure an appropriate power of 10W is available to downstream USB ports
- Include ESD protection circuitry with TVS diodes to safeguard against electrical surges
- Create an FPGA architecture capable of monitoring USB data lines in parallel and responding to malformed descriptors and keystroke injection attacks
- Provide functional FPGA controls capable of enabling or disabling downstream USB ports through switches
- Store statuses within the FPGA architecture and communicate with a microcontroller to log and relay these messages to a PC
- Use SPI communication to transmit data between the FPGA and microcontroller
- Provide basic visual feedback through on-board LEDs

Advanced Objectives:

- Include analog protection circuitry capable of continuously handling extremely high voltage transient events occurring on USB ports
- Implement monitoring of repeated enumeration or message overloading within the FPGA security architecture to prevent a denial of service
- Use scripting software to develop a visual GUI running on a PC
- Use UART communication between the PC and the microcontroller to display updates from the FPGA status visually on the GUI

Stretch Objectives:

- Include USB 3.2 high-speed data lines to enable USB 3.2 functionality
- Tune clock frequency and timing within FPGA architecture to support both USB 2.0 and USB 3.2 protocol speeds
- Implement a security parameter engine within the FPGA that can be reprogrammed via the MCU
- Include interactable GUI elements to enable user reprogramming of select security parameters

2.4 Features and Functionalities

The four-port USB hub security gateway features an on-board hardware security solution to the common attacks seen leveraging USB ports. The gateway will allow a user to connect

four downstream USB devices into their personal computer via one upstream USB port. The user will have normal functionality for any device that is connected via the USB hub. The USB hub will actively monitor USB 2.0 communication between the host device and all downstream devices. If any downstream device begins exhibiting suspicious behavior that resembles a USB attack within the threat model, the USB hub will isolate and disconnect that device from the host. This will prevent the host from experiencing a disruption or attack that a malicious USB device may be trying to enact. The USB Hub will also feature a method of allowing extra current among the 5V rail, should the connected peripherals require it. An alternative port connected via barrel jack or USB-C/micro/USB to an appropriate power source (AC Wall outlet) will allow for extra current to be delivered among the four downstream ports.

Similar forms of USB security, such as USB Sentry, leverage microcontrollers to implement security checks [5]. Our security gateway instead features a hardware-based solution driven by an FPGA architecture. Hardware based security solutions can often work in quicker time frames than a microcontroller, allowing the security gateway to catch threats more responsively. Software forms of USB protection, such as USBGuard, are unable to protect against threats exclusive to hardware [6]. Threats such as “USB Killer” attacks directly damage hardware, our system being primarily hardware security focused features analog circuitry methods of preventing electrical damage that software is unable to implement.

2.4.1 Threat Model

The assets the Four-Port USB Hub Security Gateway will protect include: the host computer, host USB data, host USB power, data from legitimate USB peripherals, gateway monitoring FPGA, USB hub/PHY hardware, power distribution circuitry, connect USB devices, and the gateway configuration parameters. The attacker is assumed to have physical access to a downstream USB port and can connect to a maliciously programmed USB device, electrically dangerous device, or a device that exploits weakness in USB communication protocols. The attacker is not assumed to have access to the internal hardware of the host device or user access to the software of the host device.

The USB Hub will protect against Malicious HID threats. These types of attacks include keyboard injections, which can lead to the compromise of the host system. The security gateway will monitor USB device communications and explicitly cut off keyboards typing at humanly impossible speeds. Device impersonation attacks (BadUSB) which lead to host compromise will be handled by descriptor and class validation within the FPGA architecture. Unauthorized devices with ambiguous descriptors will be blocked and require a manual override to be allowed through the gateway. Excessive current draw, overvoltage, ESD, and “USB Killer” events will be prevented by analog electrical circuitry present within each USB port capable of preventing electrical damage from occurring to sensitive internal circuitry. Devices causing excessive USB traffic or repeated enumeration intended to create a Denial of Service (DoS) will be blocked by the FPGA security gateway.

2.5 Engineering Specifications and Requirements

2.5.1 System Specifications and Requirements

The following table (***Table 1***) outlines the core engineering specifications of the project. The specifications highlighted in green are those that are chosen as the three demonstrable specifications. The requirements relate to the speed and performance targets of the device in terms of processing packets, making decisions, and reporting telemetry, as well as the qualifications for threat detection, the standards to be followed, and the key power and area requirements.

#	Specification	Requirement	Justification
1	USB Shutoff Latency	<= 1us upper bound, <= 100ns target	The hub must cut off connection to a port before damage occurs.
2	USB Telemetry Latency	<= 100 ms	Data must be read and reported in a timely manner.
3	HID Packets sent to host during quarantine	0 packets	Keystroke HIDs must be verified before transmitting.
4	Post-Quarantine Attack Leakage Bound	<= 1 Packet	1 packet at a max can leak through a verified device before a threat is detected.
5	Maximum Keystroke Speed	50 chars/s	A human can only type so fast, any faster is not human.
6	Device Enumeration Latency	<= 5ms	To verify keystroke input devices, the attack vector must be identified in sufficient time.
7	USB Critical Path Added Latency	<= 1ns	Added latency to a USB path can cause timing issues and missed ACKS, the architecture shall add little latency.
8	Per-port delivered Power	2.5W	Standard non power delivery USB, 5V @ 500mA.
9	USB Speed and Standard	USB 2.0 Full Speed (12 Mbps) and Low Speed (1.5 Mbps)	Support contemporary standards without significant added complexity of higher speeds like 480 Mbps or USB 3.0+.
10	Device Size	<= 165 x 115 x 40 mm (L x W x H)	The device should be reasonable but must fit in the Spartan-7 dev-board as a daughter board. This is the size for the container.
11	Total Power Draw	<= 15W	The device must be able to deliver 10W (4 x 2.5) for the 4 ports and must power the FPGA, MCU, and additional circuitry.

Table 1: Overall System Specifications

2.5.2 Component Specifications and Requirements

The following table (***Table 2***) outlines the component requirements for the project. These specifications are chosen as either reasonable bounds or as standards derived from common implementations. The speed requirement for the analog switch is derived from what is calculated as a maximum delay to prevent a second malicious packet from making its way to the USB hub, which could happen if the switch took too long to shut off. That is calculated as $t_{\text{switch_max}} = T_{\text{packet_gap}} - \left(t_{\text{FPGA_logic}}\right)$. $T_{\text{packet_gap}}$ is the minimum gap between 2 packets of 2 bits at 12Mbps, which works out to (2×83.33 ns $= 166.7$ ns), $t_{\text{FPGA_logic}}$ is the 2-clock-cycle detection pipeline delay at 48 MHz (41.7 ns), which gives an allowable turn-off threshold of approximately $t_{\text{switch_max}} \leq 120$ ns.

#	Specification	Requirement	Justification
1	Analog Switch Turn-off Speed	<= 120ns	Derived from max delay, notes above
2	USB Port Speed and Generation	USB 2.0 Full-Speed	Derived from goals of supporting a contemporary standard that is feasible.
3	FPGA Clock Speed for Sniffer	48 MHz	Standard USB oversampling rate [9].
4	MCU Clock Speed	>= 50MHz	Provides ample room for processing inputs at 10Hz to update GUI.
5	FPGA Area	> 8,000 LUTs	Plenty of room for architecture without significant congestion.

Table 2: Component Specifications

2.6 Hardware Block Diagram

Shown below is the block diagram for the hardware components of the system (***Figure 1***). It illustrates the flow from USB inputs to a clean USB output from the USB Hub integrated circuit (IC) to the external host computer with its associated GUI for monitoring the status across each channel.

Components on the PCB will be a mix of through-hole and surface mount types, and it is important to note that the FPGA will not be embedded as the surface mount IC, but rather as a development board. This choice is made because the complex circuitry and signal conditioning that an FPGA requires is outside of the scope of this project, as it requires multiple voltage levels and significant work to integrate programmable connectivity and boot devices such as the QSPI flash and JTAG connectivity that comes already integrated on the selected Digilent CMOD S7 Spartan-7 development board.

The flow starts on the left where USB inputs enter the USB ports soldered onto the main PCB. The signals exit the ports and enter analog circuitry which will perform the electrical protection against malicious USB devices which execute electrical based attacks rather than firmware-based attacks with overcurrent or overvoltage inputs. These circuitry blocks also encompass the high-speed analog switches which the FPGA controls via its kill signals (red arrows) and the current sensors which will be inputs to the MCU.

After leaving the analog circuitry, the USB packets enter the FPGA at the same time they enter the USB Hub IC. To avoid extending the critical path, the FPGA performs packet sniffing in parallel and kills further packets from entering, should a malicious pattern be detected. In some cases, this of course can let through the first packet, but many attacks such as USB Rubber Ducky HID spoofing attacks rely on payloads of multiple packets and those are the attacks we are targeting. Single packet payloads are not part of the scope here. Along with the kill signals, the FPGA interfaces with the MCU over SPI for telemetry. The MCU reads status registers inside the FPGA over that SPI interface and sends the important info over UART to the host computer, where a GUI monitors the UART bus and displays the important data to the user.

The power bus is also shown, where auxiliary power input is taken in as 5V and is then stepped down to 3.3V by means of an LDO voltage regulator. The 5V is necessary for powering the CMOD S7 FPGA development board through one of its exposed breadboardable pins, while the 3.3V is necessary for powering the MCU as well as the USB Hub IC.

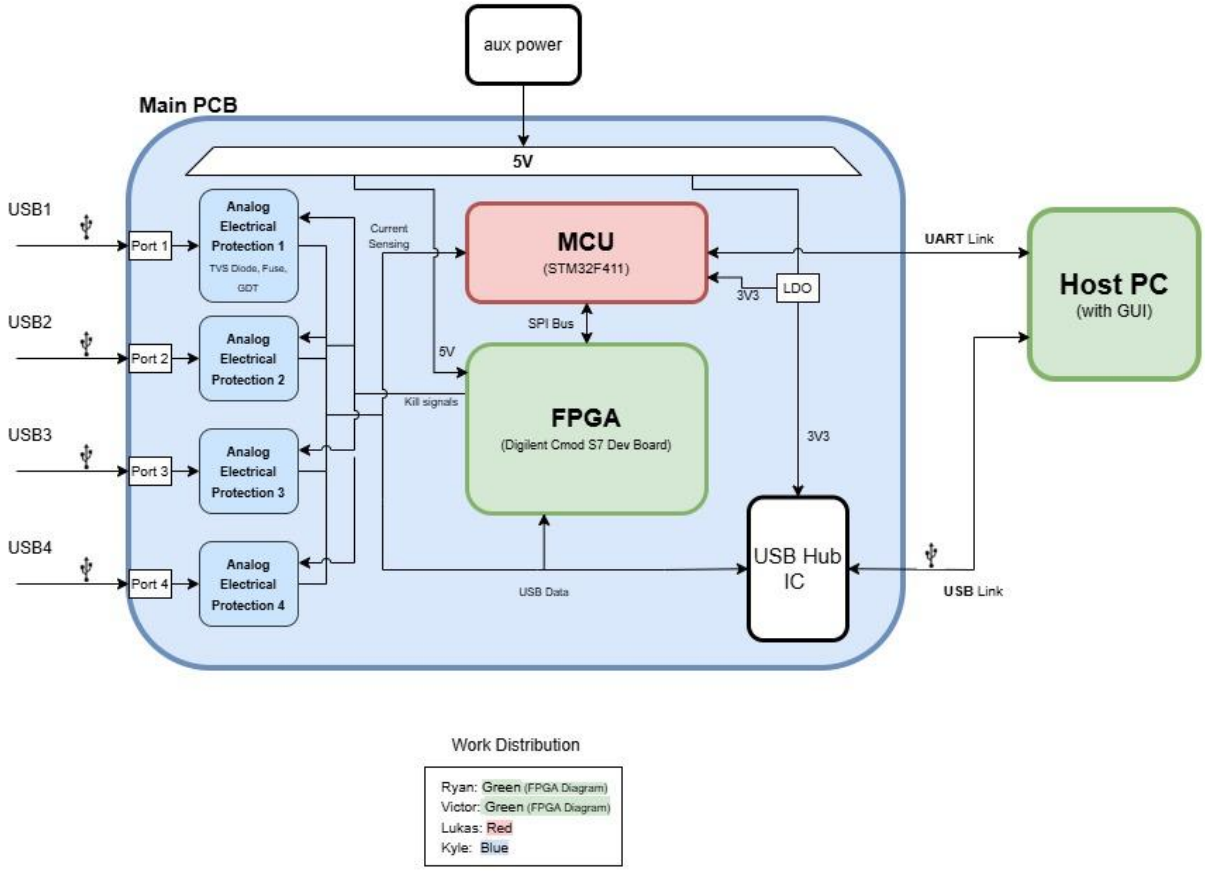


Figure 1: Main Hardware Block Diagram

2.7 FPGA Fabric Block Diagram

The FPGA has its own separate diagram (***Figure 2***), since it is neither a PCB level diagram nor a software diagram. The architecture is shown above, where a main wrapper block will hold the packet sniffer part of the logic, as well as the status registers and the SPI interface logic to connect to the MCU. The packet sniffer will also have its own wrapper, where per port sniffer blocks (all the same) will be instantiated.

The packet sniffer wrapper will connect those per port sniffer outputs to the status registers and continually update them on each clock cycle, so they are stateful. The sniffers themselves will look at incoming USB packets and their properties to search for malicious patterns, an example of which would be keystrokes that are too fast to be human. The sniffer block will include a counter to see how far apart in time each packet is, and being over a certain speed threshold will result in a channel being shut off and the status register updating to specify why (e.g. status[6] = overspeed = 1).

The SPI interface will be crucial to facilitate clean communication with the MCU, where the FPGA will be the SPI slave and will serve requests from the MCU as the master.

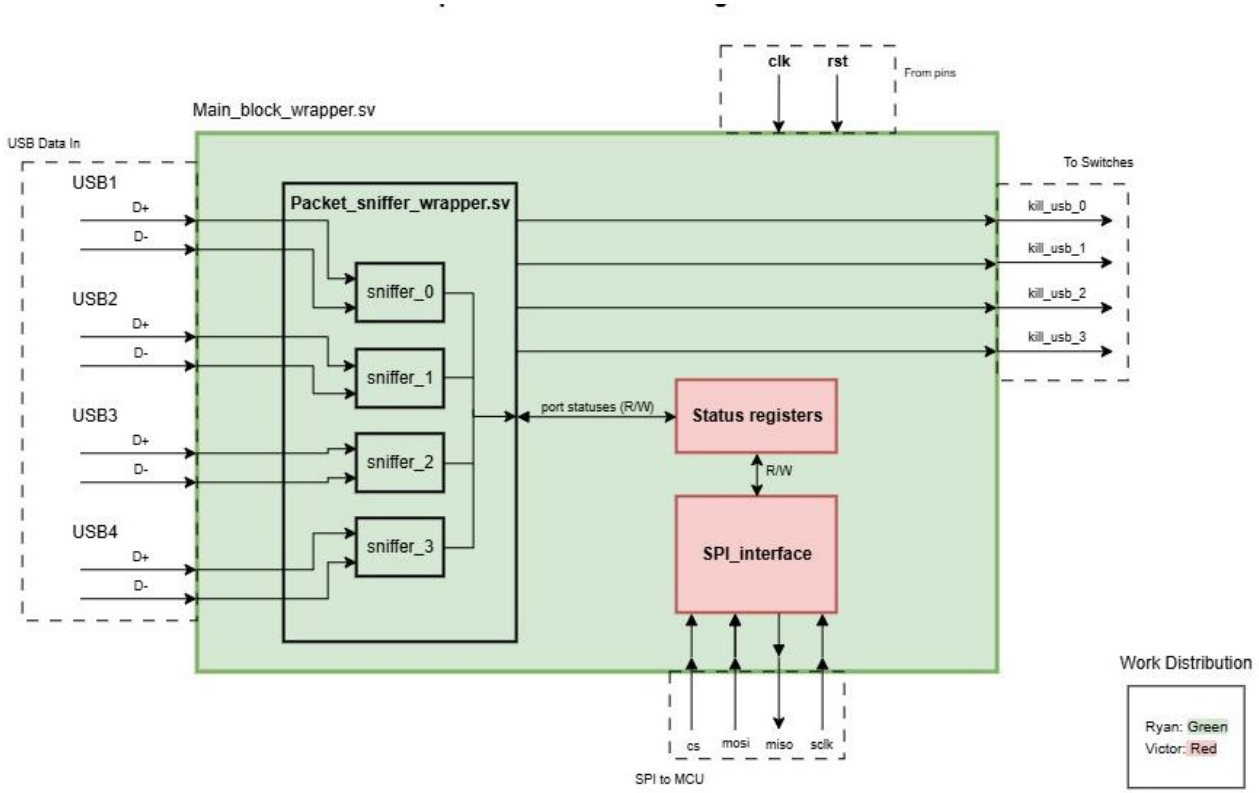


Figure 2: FPGA Block Diagram

2.8 Embedded Software Diagram

Below is the flowchart for the MCU firmware (***Figure 3***). The MCU firmware will follow a simple polling loop, since it only needs to read the data from 4 ports and from related power sensors in the analog frontend, there is not a significant need to introduce interrupt complexity or some sort of operating system. It will follow the flowchart below, in the MCU firmware box, where it continually refreshes register and sensor readings in an infinite loop and forwards that info to the GUI over UART. It will also control prompting for confirmation and receiving said confirmation that a HID keyboard identifying device is in fact a keyboard, as a first line of defense.

The GUI software will be a local application that receives serial data over UART and displays that information for the user. It will show per-port information about what threats are present, what power readings are present, and will also display confirmation prompts where necessary.

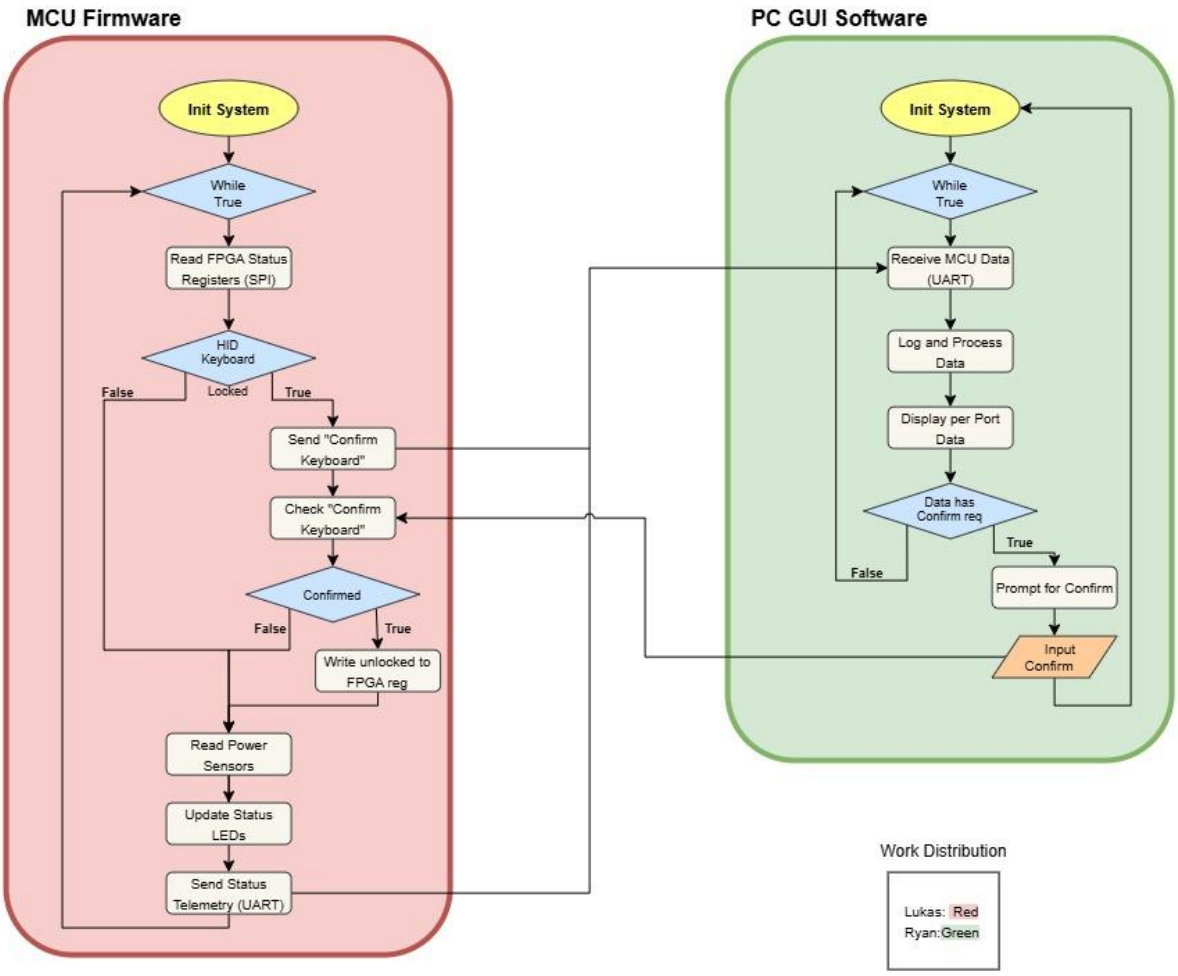


Figure 3: Software Flowchart

2.9 Prototype

To get a clearer vision of what the final product may look like, we developed a prototype figure (***Figure 4***) based on the current design shown in our hardware flowchart. The physical model consists of a compact enclosure designed to house the PCB and its associated components while providing convenient access to the USB connections. The front of the enclosure includes four downstream USB ports for connected devices. Placing these ports on the same face allows for easier access and a more organized setup for the user. The rear of the enclosure contains the upstream connections that are not as often used by the user: MCU/PC UART, power in, and the upstream USB output. Overall, the prototype was designed to provide compact and practical housing for the system while supporting the functionality outlined in our hardware design.

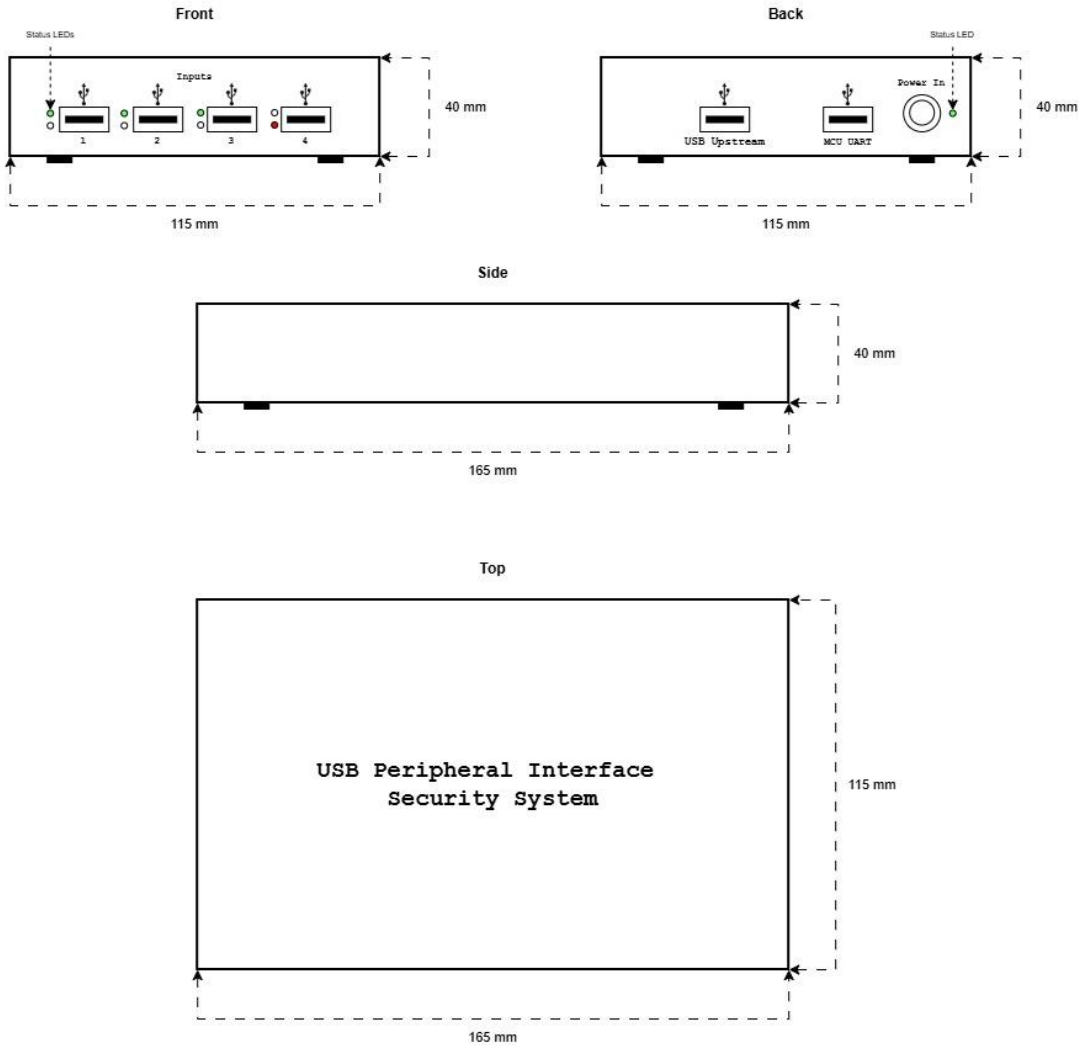


Figure 4: External Prototype Illustration

Additionally, a rough X-ray sketch of the external prototype illustration is shown below, for completeness ***(Figure 5***). It displays how the hardware on the PCB will fit into the chassis and a general idea of where components will sit.

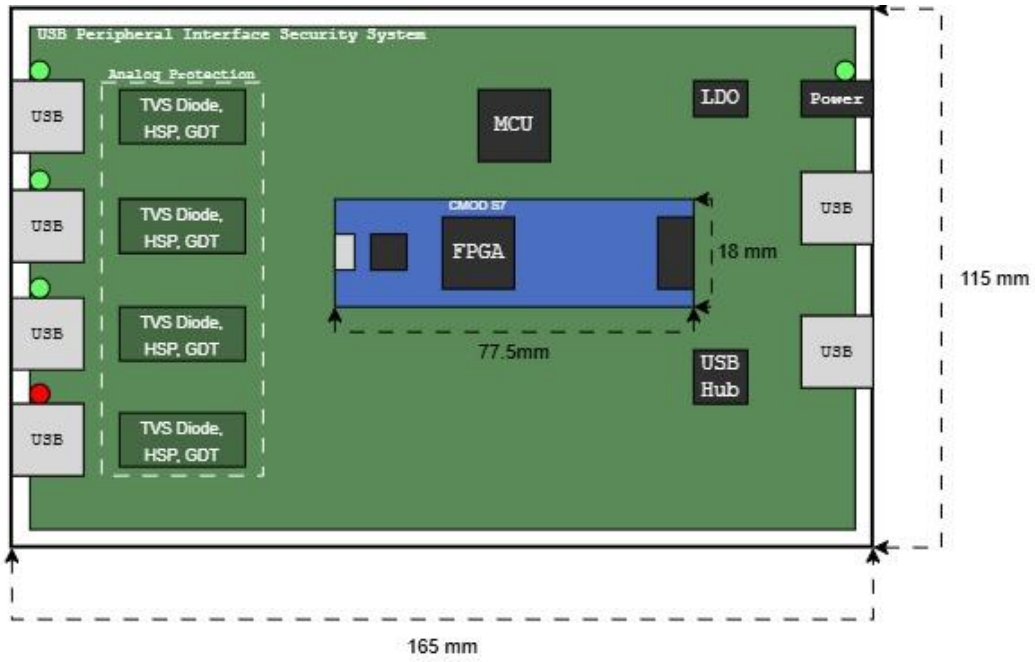


Figure 5: Internal Prototype Illustration

Finally, there is a prototype of the GUI shown below (***Figure 6***). This GUI prototype was created with the assistance of AI coding tools as a quick demonstration and is not connected to any inputs, it has dummy values to show an example of what our GUI will look like for the final product on the host PC. It shows voltage levels, current levels, as well as general port status with a log of crucial past events as a sidebar. This GUI will be displayed as a local web server address, and it will be opened in the browser.

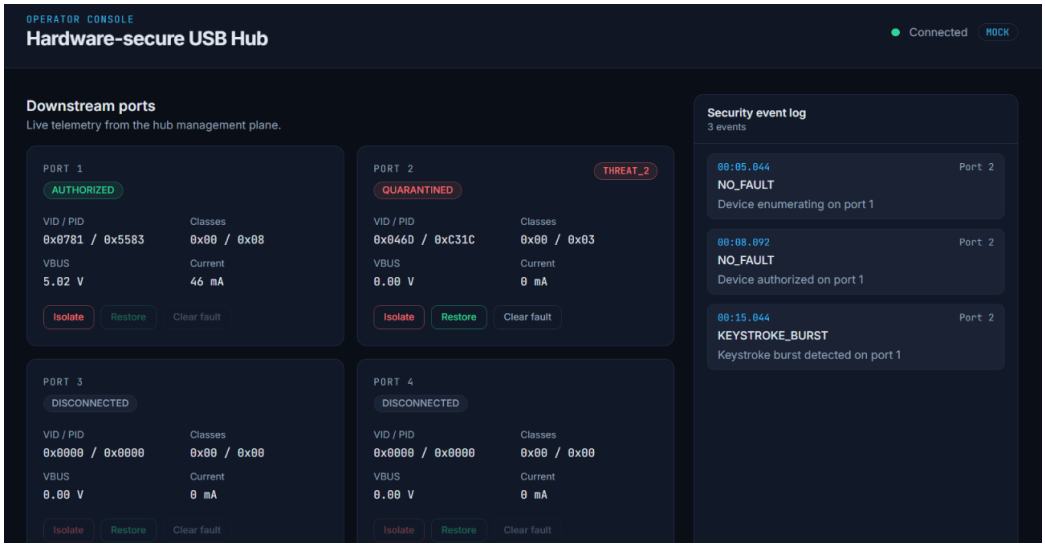
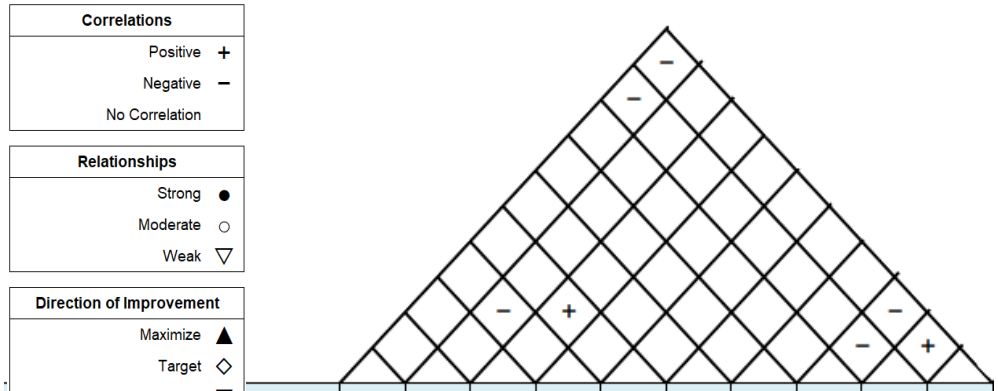


Figure 6: GUI Prototype

2.10 House of Quality

Shown is the house of quality (***Figure 7***), which represents how the customers' main requirements relate to the engineering specifications of the project. It visualizes which technical requirements have the strongest, moderate, or weakest effect on goals. The roof represents how the engineering requirements support each other or compromise, highlighting the more important design priorities.



Column #	1	2	3	4	5	6	7	8	9	10
Direction of Improvement	◇	◇	▼	▼	▼	▼	▼	◇	▼	▼
Customer Requirements \ Engineering Requirements	Downstream USB Ports	USB Speed	USB Shutoff Latency	Critical Path Added Latency	Post-Quarantine Packet Leakage	Device Enumeration Latency	USB Telemetry Latency	Per-Port Delivered Power	Total Power Draw	Device Size
Protect Host from Malicious USB Devices			●	▽	●	●	▽			
Protect Host from Electrical Attacks			●				▽	○		
Support Multiple USB Devices	●	○					○	●	○	▽
Maintain Normal USB Operation	○	●	○	●		○		○		
Respond Quickly to Threats			●	○	●	●	○			
Independently Isolate Compromised Devices	○		●		●	○	○			
Provide System Status to User							●			
Compact and Portable	▽							▽	▽	●
Provide Adequate USB Port Power	○							●	●	▽
Target	4	12 Mbps / 1.5 Mbps	≤ 100 ns	≤ 1 ns	≤ 1 packet	≤ 5 ms	≤ 100 ms	2.5 W	≤ 15 W	≤ 165 × 115 × 40 mm

Figure 7: House of Quality

2.11 Budget and Finances

Within the bill of materials (***Table 3***), expected components as well as devices necessary for the testing and demonstration of the project are included. Multiple PCB orders are listed as redesigns are expected to occur during prototyping phases and device testing. Components intended to be used within the Peripheral Interface Security System include ICs for USB hub functionality, high-speed USB data switches, USB power switches, TVS diodes, USB ports, microcontrollers, an FPGA development board, and many analog circuit components. Malicious devices are included for the purposes of providing high transient voltage events and malicious key stroke injection attacks for development and demonstration.

Item	Price	QTY.	Total Cost	Details
Digilent Cmod S7 FPGA Board	$104.00	1	$104.00	FPGA development
Main PCB	$100.00	1	$100.00	Order #1
Main PCB	$100.00	1	$100.00	Expected Redesign
USB Hub IC (CY7C65632-48AXCT)	$2.35	4	$9.40	USB 4-1 Expansion
High-Speed Analog Switches (TS3USB221ERSE)	$0.96	8	$7.68	FPGA control mechanism
STM32F411CEU6 MCU Devboard (2 ct.)	$15.99	1	$15.99	MCU for development and prototyping
STM32F411CEU6 MCU	$8.10	2	$16.20	Surface Mount MCU
USB 2.0 Type-A Port	$0.50	8	$4.00	Down and Upstream
Power Jack Connector	$0.60	2	$1.20	Supply power port
LDO Regulators	$0.15	3	$0.45	3.3V Voltage Rail
Misc. Passive Components	$15.00	1	$15.00	Resistors etc.
USB Rubber Ducky	$100.00	1	$100.00	Key Stroke Injection
USBKill Classic	$63.81	1	$63.81	Electrical Overload
TBU® High Speed Detectors (TBU-CA040-050-WH)	$1.866	20	$37.32	Protection Circuitry
TVS Diodes (CDSOD323-T05LC)	$0.809	20	$16.18	Protection Circuitry
SMT Gas Discharge Tubes (2051-09-SM)	$0.665	20	$13.30	Protection Circuitry
Multifuse® PPTC Resettable Fuse (MF-NSMF075-2)	$0.29	20	$5.80	Protection Circuitry
Total			**$610.38**	

Table 3: Bill of Materials

2.12 Milestones and Timeline

This project scope requires the use of many documents detailing the design and providing guidance for the construction of the project. This project milestones table (***Table 4***) outlines the expected key points, design completions, and prototype timeframes that are expected to be necessary in this project. Examples of dates include time periods pertaining to the design phase of the project, expected deadlines of PCB orders, and dates when testing should be completed and fully reported. Redesigns are expected and planned for within milestones, and ample time is given to ensure the final design is functional as a result of many prototyping phases.

Milestone	Timeline	Status
Project Proposal Document	09/11/26	Complete
Revised Project Proposal Document	09/18/26	Awaiting Feedback
Detailed Concept Design	09/25/26 – 10/09/26	Not Started
Baseline Analysis	10/09/26 – 11/13/26	Not Started
Electrical Breadboard Prototype	10/16/26 – 11/13/26	Not Started
Prototype Code	10/16/26 – 11/13/26	Not Started
PCB Order #1	11/06/26	Not Started
Prototype Assembly and Testing	11/13/26 – 11/27/26	Not Started
Testing Report	11/27/26	Not Started
Redesign Analysis	11/27/26 – 12/11/26	Not Started
PCB Order #2	12/11/26	Not Started
Prototype Assembly and Testing	1/11/27-1/25/27	Not Started
Final Redesign and Order	1/25/27-2/15/27	Not Started
Final Design Assembly and Testing	2/15/27-3/8/27	Not Started
Final Testing Demo	4/04/27	Not Started
Final Paper	12/03/26	Not Started

Table 4: Milestones and Timeline

2.13 Risk Analysis and Mitigation

Developing the USB security hub comes with several technical and project risks that could impact performance and/or push back the project timeline. With the main risks being the complexity of the FPGA design, reliable communication between the MCU and FPGA, meeting timing and telemetry requirements, monitoring multiple USB ports at once, and overall system integration.

To mitigate these risks, we will test key subsystems early instead of waiting until the full system is complete. The MCU-to-FPGA interface will be verified ahead of time before it is integrated with the rest of the design. The system will also be integrated in stages, so problems can be found and addressed earlier. The table below outlines these risks (***Table 5***).

Risk	Impact	Mitigation
FPGA development takes longer than expected	Delays integration	Prototype FPGA functions early
MCU-FPGA communication issues	Delays control/status integration	Test SPI interface separately
FPGA USB monitoring does not meet timing requirements	Security response may not meet specifications	Test FPGA timing and shutoff logic early
MCU telemetry is too slow	System status may not meet telemetry requirements	Test UART communication and optimize firmware polling
Four-port monitoring increases FPGA complexity	Could delay multi-port implementation	Develop and verify one port first before expanding to more
FPGA resource usage exceeds expectations	FPGA design may not fit or meet timing	Monitor register usage throughout development and optimize logic
PCB requires another revision	Schedule and cost delay	Thorough design review before ordering
Power budget is insufficient	Ports cannot operate as intended	Calculate worst-case power early
Integration and testing take longer than expected	Final prototype delayed	Integrate subsystems progressively
Components become unavailable	Hardware assembly may be delayed	Order important components early and find alternatives if needed

Table 5: Risk Analysis

3 Appendices

3.1 Appendix A – References

[1] GAM Tech Team, "What Is a USB Killer? How It Works + How to Protect Devices (2026)," GAM_Tech_CA. Accessed: Sep. 10, 2026. [Online]. Available: https://www.gamtech.ca/category/blog/what-is-a-usb-killer

[2] M. Tischer, "Users Really Do Plug in USB Drives They Find," *2016 IEEE Symposium on Security and Privacy (SP)*, pp. 306–319, 2016, doi: 10.1109/SP.2016.26.

[3] *Universal Serial Bus Specification*, Revision 2.0, USB Implementers Forum, Apr. 2000. [Online]. Available: https://www.usb.org/document-library/usb-20-specification

[4] B. Ben Atar, "USB Attacks and Prevention," Sepio. Accessed: Sep. 10, 2026. [Online]. Available: https://sepiocyber.com/resources/use-cases/preventing-advanced-usb-attacks/

[5] Architecture Technology Corporation, "USB Sentry - Architecture Technology Corporation," Atcorp.com. Accessed: Sep. 10, 2026. [Online]. Available: https://www.atcorp.com/r-and-d/usbsentry/

[6] S. Pipping, "Rule Language | USBGuard," Github.io. Accessed: Sep. 10, 2026. [Online]. Available: https://usbguard.github.io/documentation/rule-language

[7] R. Fisk, "The USB Security Gateway Is Good," Github-wiki-see.page. Accessed: Sep. 10, 2026. [Online]. Available: https://github-wiki-see.page/m/robertfisk/USG/wiki

[8] Bourns, "Bourns | USB Kill Threats," Bourns.com. Accessed: Sep. 10, 2026. [Online]. Available: https://bourns.com/resources/technical-library/library-documents/circuit-protection-technical-library/protect-mobile-devices-from-usb-kill-threats

[9] Silicon Labs. *AN0046: USB Hardware Design Guidelines*. Rev. 1.2, Silicon Laboratories, 2020, https://www.silabs.com/documents/public/application-notes/an0046-efm32-usb-hardware-design-guidelines.pdf

3.2 Appendix B – Copyright Permission

3.3 Appendix C – Etc.